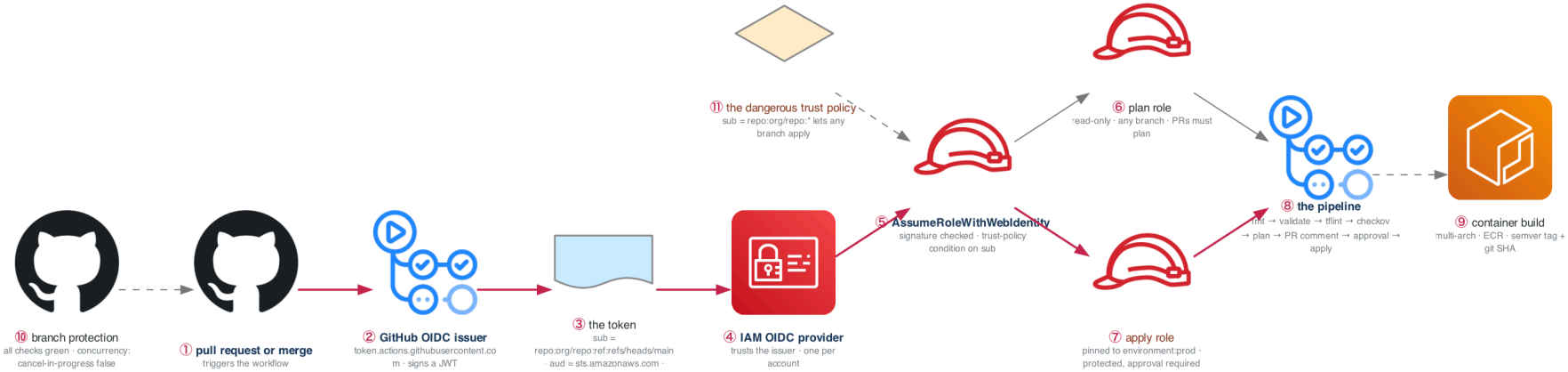
OIDC: how a workflow gets AWS credentials with no secret anywhere, numbered

Week 12 · Figure 12.1 · figure with badges 1–11 · 2026-10-17



What to say, badge by badge

About three and a half minutes at a normal pace (425 words). Point at each badge as you say its line.

Open
“Three hours of work this week read as three years of experience, and this is the picture of it. No access key exists. A workflow proves who it is with a signed token, and AWS decides what that identity may do. Follow the blue path, then look at the two roles.”

1 pull request or merge
“The workflow starts with a context: which repository, which branch or environment, which event. That context is about to become the identity.”

2 GitHub OIDC issuer
“GitHub runs an OpenID Connect issuer. On request, it signs a JSON Web Token with its private key and publishes the public key at a well-known URL. Anyone can verify; only GitHub can sign.”

3 the token
“The claims are the whole story. `sub` says repo, and ref or environment. `aud` says who the token is for: sts.amazonaws.com. It expires in minutes. There is nothing in it worth stealing for long.”

4 IAM OIDC provider
“One per AWS account, pointing at GitHub's issuer. This is AWS agreeing to check signatures from that issuer. It grants nothing by itself.”

5 AssumeRoleWithWebIdentity
“The workflow hands the token to STS. STS verifies the signature against the published key, then evaluates the role's trust policy against the claims. The condition on `sub` is the access control. STS returns temporary credentials, and the job runs.”

6 plan role
“Read-only. Trusted for any branch, because a pull request from any branch must be able to produce a plan. Read-only means a malicious PR can look but not touch.”

7 apply role
“Trusted only for `environment:prod`, a GitHub environment with required reviewers. Merging is not enough; a human approves the deployment, and only then does a token with that claim exist.”

8 the pipeline
“fmt, validate, tflint, checkov, plan, a comment on the PR with the plan, approval, apply. Cheap checks first, so a formatting error does not spend a plan. The plan comment is what the reviewer actually reads.”

9 container build
“Multi-arch, pushed to ECR with two tags: a semver for humans and the git SHA for provenance. Week 18 signs the digest; this is where the digest is born.”

10 branch protection
“Every check green before merge. The concurrency group has cancel-in-progress false, because cancelling an apply halfway through is how you get a half-built environment.”

11 the dangerous trust policy
“`repo:org/repo:*` matches every branch and every PR. On the plan role it is fine. On the apply role it means anyone who can push a branch can deploy to production. Say this in the interview; it is the question they ask.”

Close
“So: no key exists, identity is a signed claim, and the claim decides the role. The exit criterion is a grep for AKIA that returns nothing, in the tree and in the history.”